Technical Documentation — NewsBot Intelligence System 2.0

Technical architecture, reproducible methods, evaluation, and deployment.

Purpose and architecture

NewsBot 2.0 turns the completed midterm NewsBot into a modular local research platform. The architecture separates data handling, analysis modules, language-model adapters, multilingual utilities, conversation, web delivery, and evaluation. The public orchestrator is `NewsBot2IntegratedSystem`; it initializes the reusable components once, fits them on the local corpus, and returns JSON-serializable evidence for an entered article or a corpus query.

```
processed CSV → validation → preprocessing/features → classifier/topic/search indexes
■ sentiment / NER / relationships
entered text or query → integrated system → component-level results + warnings
■ Streamlit / Flask / notebooks / exports
```

Components intentionally fail independently. For example, an unavailable transformer adds a warning and uses the extractive fallback; classification, sentiment, topics, and NER still run. This is a practical reliability choice, not a claim that the fallback equals a pretrained model.

Data and preparation

`src/data_processing/data_validator.py` loads the preserved 1,800-row, six-category HuffPost sample and normalizes dates, titles, authors, and `full_text`. Validation checks required columns, missingness, duplicates, category counts, and dates. `TextPreprocessor` carries forward the midterm cleaning approach before TF-IDF feature construction. The project treats headline-plus-description records as short news records, not full articles. Authored Spanish/French examples and evaluation sets remain separate in `data/demo/`.

Models and algorithms

`AdvancedNewsClassifier` compares the midterm Multinomial Naive Bayes baseline with enhanced Logistic Regression and calibrated Linear SVM candidates on a deterministic stratified holdout. It selects by macro F1, returns probabilities/alternatives, and calculates held-out accuracy, macro precision/recall/F1, weighted F1, confusion matrix, and top-label expected calibration error. Linear feature explanations are available only when Logistic Regression is selected.

`TopicDiscoveryEngine` trains both CountVectorizer/LDA and TF-IDF/NMF with fixed random state. It exports top words, document-topic distributions, annual trends, emerging/declining topic IDs, and plots. Topic diversity and lexical overlap are useful diagnostics, but the reported “coherence proxy” is not human semantic coherence; notebooks include qualitative interpretation.

`SentimentEvolutionTracker` uses VADER’s transparent compound thresholds, TextBlob subjectivity when installed, a small documented emotion lexicon, and annual/category aggregates. `EntityRelationshipMapper` uses spaCy NER plus transparent local co-occurrence/dependency heuristics and exports a NetworkX GraphML file. Edges are corpus mentions, not proven real-world relationships.

`IntelligentSummarizer` implements a lazy `sshleifer/distilbart-cnn-12-6` integration with CUDA detection and deterministic extractive fallback. `SemanticSearchEngine` implements the lazy multilingual `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2` path with NumPy cosine search and a TF-IDF fallback. Runtime responses disclose the active backend. `NEWSBOT_ENABLE_TRANSFORMERS=1` activates downloads; the default is intentionally safe for local test execution.

The multilingual adapter detects language with deterministic `langdetect`, supports authored Spanish/French demonstrations, attempts cached MarianMT before an optional network translator, and returns availability/provenance. Cross-language comparisons translate only when available and state that they are coverage/framing indicators rather than cultural conclusions.

Public API and configuration

Key public APIs are documented in `docs/api_reference.md`. The principal methods are `fit`, `comprehensive_analysis`, `batch_analysis`, `query_interface`, and `generate_insights_report`. Conversation parses category, sentiment, count, capitalized entities, comparisons, topic phrases, and historical “this week/month” windows relative to the corpus maximum date. Its deterministic response templates cite local title/ID records and never fabricate source content.

`NewsBot2Config` controls seed, corpus paths, category set, topic count, batch size, confidence threshold, model names, translation backend, and transformer flags. Secrets are read only from environment variables. Useful variables are `NEWSBOT_SECRET_KEY`, `NEWSBOT_MAX_BATCH_SIZE`, `NEWSBOT_ENABLE_TRANSFORMERS`, `NEWSBOT_TRANSFORMERS_LOCAL_ONLY`, and `NEWSBOT_TRANSLATION_BACKEND`.

Evaluation and performance

Run `./.venv/bin/python scripts/run_phase2.py` to regenerate all evidence. It produces model comparison, topic quality, summary, semantic-search, multilingual, conversation, sentiment, and data-validation tables plus `data/results/metrics/evaluation_summary.json`. The default evaluated run is honest about its active fallback backends; it does not label TF-IDF output as transformer embeddings. The author-created evaluation sets cover six retrieval queries, six multilingual pairs, one non-leaking summary reference, and nine conversation cases. They are small demonstrations, not production benchmarks.

The recorded baseline result is 0.714 accuracy and 0.712 macro F1. The balanced sample makes macro metrics appropriate. Retrieval evaluation reports category-based Precision@1/5 and Hit@5; it measures topical relevance, not factual accuracy. Summarization reports compression, Flesch reading ease, entity preservation proxy, and ROUGE-L where a clean authored reference exists.

Installation, deployment, and security

Create a virtual environment, install pinned requirements, install `en_core_web_sm`, run tests, and then generate artifacts. Streamlit (`streamlit_app.py`) is the primary bonus dashboard; Flask routes provide `/`, `/analyze`, `/batch`,

`/query`, `/api/analyze`, `/api/query`, and `/health`. `Dockerfile`, `Procfile`, request-size limits, Jinja autoescaping, and an environment-sourced secret support basic deployment hygiene. See `docs/deployment_guide.md` for commands.

No `.env`, raw source corpus, or model checkpoint is committed. Inputs are capped, batch inputs are limited, and the application returns safe error messages. This project is not a fact-checking service, a live-news source, or a system for high-stakes automated decisions.